```
# ============================================================
# SHAKESPEARE TEXT GENERATION USING DEEP LSTM
# Dataset: Shakespeare Corpus
# Framework: TensorFlow / Keras
# ============================================================

import os
import urllib.request
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# -------------------- 1. CONFIGURATION --------------------

SEED = 42
tf.keras.utils.set_random_seed(SEED)

SEQ_LENGTH = 100
STEP = 3
EMBEDDING_DIM = 128
LSTM_UNITS = 256
BATCH_SIZE = 128
EPOCHS = 15

print("TensorFlow version:", tf.__version__)
print("GPU available:", tf.config.list_physical_devices("GPU"))

# -------------------- 2. DOWNLOAD SHAKESPEARE CORPUS ----------

DATA_URL = (
    "https://storage.googleapis.com/download.tensorflow.org/"
    "data/shakespeare.txt"
)
DATA_PATH = "/content/shakespeare.txt"

if not os.path.exists(DATA_PATH):
    urllib.request.urlretrieve(DATA_URL, DATA_PATH)

with open(DATA_PATH, "r", encoding="utf-8") as f:
    text = f.read()

print("\nCorpus downloaded successfully.")
```

```
print("\nCorpus downloaded successfully.")
print("Total characters:", len(text))
print("\nCorpus preview:\n")
print(text[:500])

# -------------------- 3. TEXT PREPROCESSING --------------------

text = text.lower()

vocab = sorted(set(text))
char_to_idx = {char: idx for idx, char in enumerate(vocab)}
idx_to_char = np.array(vocab)

vocab_size = len(vocab)

encoded_text = np.array(
    [char_to_idx[char] for char in text],
    dtype=np.int32
)

print("\nVocabulary size:", vocab_size)
print("Encoded corpus length:", len(encoded_text))

# -------------------- 4. CREATE TRAINING SEQUENCES --------------

sequences = tf.keras.utils.timeseries_dataset_from_array(
    data=encoded_text[:-1],
    targets=encoded_text[SEQ_LENGTH:],
    sequence_length=SEQ_LENGTH,
    sequence_stride=STEP,
    shuffle=False,
    batch_size=None
)

# Convert dataset into batched inputs and next-character targets.
dataset = sequences.batch(BATCH_SIZE, drop_remainder=True)

dataset = dataset.shuffle(
    10000,
    seed=SEED,
    reshuffle_each_iteration=True
).prefetch(tf.data.AUTOTUNE)

print("\nTraining sequences prepared.")

# -------------------- 5. BUILD DEEP LSTM MODEL ------------------

model = Sequential([
    Embedding(
```

```
        input_dim=vocab_size,
        output_dim=EMBEDDING_DIM
    ),

    LSTM(
        LSTM_UNITS,
        return_sequences=True
    ),

    Dropout(0.2),

    LSTM(
        LSTM_UNITS
    ),

    Dropout(0.2),

    Dense(vocab_size)
])

model.compile(
    optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(
        from_logits=True
    ),
    metrics=["sparse_categorical_accuracy"]
)

model.build(input_shape=(None, SEQ_LENGTH))
model.summary()

# -------------------- 6. TRAIN THE MODEL --------------------

early_stopping = EarlyStopping(
    monitor="loss",
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    dataset,
    epochs=EPOCHS,
    callbacks=[early_stopping],
    verbose=1
)

# -------------------- 7. PLOT TRAINING PERFORMANCE --------------

plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)
plt.plot(history.history["loss"], marker="o")
plt.title("LSTM Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Cross-Entropy Loss")
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(
    history.history["sparse_categorical_accuracy"],
    marker="o"
)
plt.title("Next-Character Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# -------------------- 8. GENERATE TEXT --------------------

def generate_text(seed_text, length=1000, temperature=0.8):
    seed_text = seed_text.lower()

    if not seed_text:
        seed_text = "to be or not to be"

    seed_text = "".join(
        char if char in char_to_idx else " "
        for char in seed_text
    )

    generated = seed_text

    for _ in range(length):
        input_text = generated[-SEQ_LENGTH:]

        encoded = [
            char_to_idx.get(char, char_to_idx[" "])
            for char in input_text
        ]

        if len(encoded) < SEQ_LENGTH:
            encoded = (
                [char_to_idx[" "]] * (SEQ_LENGTH - len(encoded))
                + encoded
            )
```

```
            )

        input_array = np.array(
            [encoded],
            dtype=np.int32
        )

        predictions = model(
            input_array,
            training=False
        )[0]

        predictions = predictions / temperature

        next_char_idx = tf.random.categorical(
            predictions[tf.newaxis, :],
            num_samples=1
        )[0, 0].numpy()

        next_char = idx_to_char[next_char_idx]
        generated += next_char

    return generated

print("\n========== GENERATED SHAKESPEARE TEXT ==========\n")

generated_text = generate_text(
    seed_text="to be or not to be",
    length=1000,
    temperature=0.8
)

print(generated_text)

# -------------------- 9. COMPARE DIFFERENT TEMPERATURES ---------

temperatures = [0.4, 0.8, 1.2]

for temp in temperatures:
    print("\n" + "=" * 60)
    print("TEMPERATURE:", temp)
    print("=" * 60)

    print(
        generate_text(
            seed_text="the king",
            length=300,
            temperature=temp
        )
```

```
    )

# -------------------- 10. EVALUATE USING PERPLEXITY --------------

# Character-level perplexity on a held-out section.
split_index = int(len(encoded_text) * 0.9)

eval_text = encoded_text[split_index:]

eval_sequences = tf.keras.utils.timeseries_dataset_from_array(
    data=eval_text[:-1],
    targets=eval_text[SEQ_LENGTH:],
    sequence_length=SEQ_LENGTH,
    sequence_stride=SEQ_LENGTH,
    shuffle=False,
    batch_size=128
)

loss_metric = tf.keras.metrics.Mean()

for x_batch, y_batch in eval_sequences:
    logits = model(x_batch, training=False)

    targets = y_batch

    loss = tf.keras.losses.sparse_categorical_crossentropy(
        targets,
        logits,
        from_logits=True
    )

    loss_metric.update_state(loss)

eval_loss = float(loss_metric.result().numpy())
perplexity = float(np.exp(min(eval_loss, 50)))

print("\n========== EVALUATION RESULTS ==========")
print(f"Evaluation cross-entropy loss: {eval_loss:.4f}")
print(f"Character-level perplexity: {perplexity:.4f}")

# -------------------- 11. BASIC TEXT QUALITY INSPECTION ---------

def text_quality_report(generated):
    words = generated.split()
    unique_words = set(words)

    repeated_chars = sum(
        generated[i] == generated[i - 1]
        for i in range(1, len(generated))
```

```
    )

    print("\n========== TEXT QUALITY INSPECTION ==========")
    print("Generated characters:", len(generated))
    print("Word count:", len(words))
    print(
        "Unique-word ratio:",
        round(len(unique_words) / max(len(words), 1), 4)
    )
    print(
        "Repeated adjacent character ratio:",
        round(repeated_chars / max(len(generated) - 1, 1), 4)
    )
    print("\nGenerated sample:\n")
    print(generated[:500])

    print(
        "\nInterpretation: inspect grammar, sentence structure, "
        "word choice, and Shakespeare-like style manually. "
        "These simple statistics do not measure semantic coherence
    )

text_quality_report(generated_text)

# -------------------- 12. SAVE MODEL AND GENERATED TEXT ---------

model.save("/content/shakespeare_lstm.keras")

with open(
    "/content/generated_shakespeare.txt",
    "w",
    encoding="utf-8"
) as f:
    f.write(generated_text)

print("\n========== PROJECT COMPLETED ==========")
print("Saved model: /content/shakespeare_lstm.keras")
print("Saved generated text: /content/generated_shakespeare.txt")
```

```
TensorFlow version: 2.21.0
GPU available: []

Corpus downloaded successfully.
Total characters: 1115394

Corpus preview:

First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

All:
We know't, we know't
```